When Deleted Agent Memories Return: A Controlled Study of Durable Scope Revocation

EZEKIEL OLOGUNDE

Deleting a memory record does not ensure that an agent cannot recover or reuse its contents. We evaluate a project-scope revocation overlay combining materialized lineage, an independently current authority, commit validation, and restore reconciliation. A controlled study uses 140 corpus-grounded episodes, two persistent reference backends, six configurations, and 15,680 lifecycle and boundary trials. The full overlay yields zero active-record, persisted-file, or retrieval recoveries in 2,520 managed trials; one-time dependency purge fails after six of nine event schedules. Authority rollback and interception bypass defeat the overlay, and retained replay copies remain recoverable. A separate 24-episode experiment executes 744 primary local-model inferences plus 48 exploratory evidence-swap probes. Guarded restoration with fresh caller context matches the never-seen control by evidence construction, whereas caller-retained summaries preserve behavioral influence despite clean managed storage. A process-cache audit finds target-containing memoization keys in all 48 original-overlay cases; explicit cache clearing removes those reachable keys in all 48 matched cases. These results distinguish durable active-use revocation, recoverable process copies, and supplied-context influence. They do not establish model-weight forgetting, physical zeroization, or deployment-wide erasure. The contribution is an auditable bounded systems study, with explicit failure assumptions and reproducible artifacts.

Additional Key Words and Phrases: agent memory, data remanence, scope revocation, deletion durability, fault injection

1 INTRODUCTION

An instruction to delete a project's memory can involve more than removing the original record. An external-memory pipeline may store a transcript, a compressed summary, a cached response, an index payload, and a checkpoint. Even after these active artifacts are removed, an old snapshot or a queued writer can return the information. The system may acknowledge deletion correctly at one instant while failing to preserve that state through later operations.

This paper investigates a narrow systems question: whether scope-level revocation can remain effective under explicitly controlled lifecycle transitions. It does not attempt to prove that an AI system has forgotten everything about a subject. It separates active-artifact recoverability, retrieval visibility, retained-copy recoverability, and model behavior. The first study measures active artifacts and retained copies. A second study executes a local model with generated summaries and probes for explicit emission and route influence; a separate process-cache audit extends forensic coverage. Model weights are fixed and semantic embedding inversion is not attempted.

Previous work already studies derived-memory residue, counterfactual behavior, mutation-time hooks, and reinjection prevention. Accordingly, neither tombstones nor provenance are claimed as new mechanisms. Our contribution is a reproducible stress protocol that makes restore and write-path assumptions observable, supported by persistent implementations and deliberately failing controls. The result is a bounded systems study with a small local-model extension; integration with production memory frameworks remains outside its validation.

We ask: *Under pre-deletion snapshot replay and delayed-write interleavings, does a project-scoped overlay reduce active recoverability compared with source deletion and a one-time dependency purge, while preserving unrelated memory?* Our implementation uses an independently current authority: ordinary memory snapshots do not include or replace its state. The deliberately violated-authority case establishes why that assumption matters.

Author's address: Ezekiel Ologunde.

The study contributes (1) a scoped definition of durable revocation and corresponding state-transition tests; (2) two persistent reference adapters and a configurable overlay; (3) six-arm, two-backend measurements with component ablations; and (4) a result bundle that retains raw trial outcomes, protocol and code hashes, data provenance, and limitations. The extension additionally measures reachable memoizer keys and actual generated responses under six paired evidence conditions. It does not claim field-wide priority or independent replication of a deployed framework.

2 RELATED WORK AND POSITIONING

2.1 External memory and deletion evaluation

Chen et al. [1] examine raw and summary tiers with injected canaries and a forgetting-residue measure. Their versioned HTML describes a 50-instance deletion evaluation and explicitly identifies meaningful facts and additional storage surfaces as extensions. Our study complements that output-oriented measurement with persistent-file inspection and lifecycle transitions. It does not reproduce their model results.

Yang [2] compares control-plane placement using ForgetEval, including a released adversarial suite and memory adapters. Its limitations discuss substring-scoring artifacts and uneven system capabilities. Mutation-time interception is therefore established prior work. Our contribution concerns the durability contract when old state or old authorization returns, rather than a new semantic deletion planner.

Liu and Li [3] describe MemScope, paired counterfactual worlds, sham controls, and dependency-aware purge in their publicly indexed abstract. We treat behavioral counterfactual forgetting as overlapping prior work. Because full methods were unavailable during this pass, we do not reproduce MemScope or compare its numerical effects with our independently specified local-model experiment.

Wang and Zhang [4] distinguish retained correlated information from directly deleted entries in multimodal memory. Such inference is a separate channel: removing linked records does not imply eliminating every fact that can be inferred from legitimate retained context. Our exact synthetic targets diagnose copy reappearance and do not establish semantic non-inference.

2.2 Lifecycle security and model-side forgetting

The DSIT-commissioned review [5] motivates end-of-lifecycle research, but its January 2026 literature cutoff cannot establish the novelty of later agent-memory work. Chourasia and Shah [6] analyze deletion guarantees in machine learning. Ding et al. [7] study influence regeneration during continued training in federated agent networks. Our frozen-model storage setting avoids interpreting external-store cleanup as model unlearning.

Shen [8] uses read-time evidence restoration to diagnose memory eviction. That intervention is distinct from restoring a pre-deletion storage snapshot. The common methodological lesson is to define exactly what restoration changes; an accuracy difference alone does not identify a storage mechanism.

2.3 Execution state, structural forgetting, and deployed APIs

Yao et al. [13] already reconstruct agent execution state through provenance-guided sanitized replay and compare behavioral outcomes with a clean counterfactual. Their result covers reconstructible runtime state under explicit assumptions, rather than model parameters or committed side effects. We therefore claim neither the first behavioral deletion audit nor the first provenance-based runtime cleanup. Our different operational emphasis is persistent revocation authority, stale admission, and snapshot resurrection; we do not reproduce their KV reconstruction.

Washio [14] proposes structural forgetting through multiple gating and revocation mechanisms. Only the indexed abstract/excerpt was accessible, so a full-method comparison is not asserted. Gaggar [15] evaluates policy-governed lifecycle management on a synthetic enterprise workload. That study already includes deletion completeness; we emphasize separately inspectable transitions and boundaries rather than treating a completeness percentage as universal erasure.

Lee's Temvera artifact [16] distinguishes deletion requested in prose from native API deletion and inspects residual stores. Its source explicitly cautions against comparing a typed purge with an English instruction. Our source-only reference control is likewise deliberately weak: it cannot support a claim that a vendor's native API fails. The one-time dependency-purge arm provides our meaningful cleanup comparator.

Chen et al.'s MemSecBench [18] already evaluates memory poisoning through persistence, downstream consequences, and selective repair using checkpoint-specific evidence. Its authors explicitly separate repair from irrecoverability and exclude inaccessible caches from their snapshot audit. Lifecycle evaluation and benign-memory preservation are therefore prior capabilities. Our narrower contribution is the specified persistent-store stress grid and directly reachable memoizer-key audit; we do not reproduce MemSecBench's harness/backend comparison or claim a new lifecycle framework.

2.4 Novelty boundary

The reviewed literature already establishes cross-layer cleanup, behavioral controls, and governed lifecycles. The remaining question here is the behavior of a specified persistent-store contract and its uncovered copy boundaries, rather than a demonstrated absence of such ideas in prior work. Our candidate contribution combines a bounded persistent-store audit, controlled stale-writer and snapshot-replay events, and explicit authority/bypass negative controls. This is a scoped engineering contribution. A claim that this is the first such system would require broader searches and complete comparison with overlapping frameworks, which are not established by this experiment.

3 SYSTEM MODEL AND DELETION CONTRACT

3.1 Artifacts, scopes, and the current authority

Let an artifact be $a = (i, \ell, c, P, G)$, where i is an opaque identifier, ℓ its storage layer, c its content, P its parent identifiers, and G a mapping of scope identifiers to the generations under which it was created. A record may depend on more than one scope. The prototype stores materialized scope membership on every derivative; its purge selects that membership directly. It does not infer or traverse an unknown dependency graph.

For each scope s, the authority stores $D(s) = (e_s, r_s)$, with generation e_s and a revocation flag r_s. A never-revoked scope defaults to $(0, \text{false})$. Artifact admission requires

$$\text{admit}(a, D) = \bigwedge_{s \in \text{dom}(G_a)} [\neg r_s \ \wedge \ G_a(s) = e_s]. \tag{1}$$

Deletion advances the scope generation and marks it revoked. The first prototype has no reauthorization API: a legitimate new disclosure would require an explicitly specified policy and new generation assignment. Blocking an old scope indefinitely is not presented as a complete product policy.

The authority is stored separately from the memory snapshot. This is an assumption of the protocol, not an intrinsic property of JSON files. Atomic replacement protects file-update structure under tested operation; the prototype does not implement a distributed transaction, power-loss synchronization guarantee, or hardware rollback-resistant counter.

3.2 Operations

The full overlay performs four actions. First, it records revocation. Second, it removes active artifacts carrying the target scope, including the source and materialized derivatives. Third, every managed writer validates its artifact generation when committing. Fourth, restore filters snapshot contents against the current authority before publishing them into the active store. The materialized index is rebuilt from the resulting records.

The test coordinator serializes authority checks and writes. A check followed by an unprotected concurrent write in a production system can still race with revocation. Production integration would require a transaction, lock, or compare-and-swap spanning authorization and publication. Our controlled delayed-write schedule verifies the decision at the commit boundary; it is not a proof of concurrent linearizability.

3.3 Success and boundaries

Active revocation succeeds for a tested event if the synthetic target is absent from the active record content and its materialized index payload. Persisted-file absence is measured separately by scanning all currently reachable files in the active adapter directory. Those scans include SQLite database, WAL/shared-memory files when present, and the index sidecar, or JSON record/index files. They do not inspect released filesystem blocks, operating-system page caches, SSD firmware, remote telemetry, or all physical media.

A retained snapshot is a different object. Our fault-replay fixture deliberately retains the pre-deletion state so that restoration is testable. It remains outside active purge and is explicitly scored as recoverable. An effective restore gate can prevent that old content from becoming active while its retained bytes still exist. Therefore the study establishes scoped revocation, not whole-system erasure.

4 IMPLEMENTATION

4.1 Persistent adapters

The SQLite adapter stores JSON artifact payloads in a relational table and maintains an FTS5 table. It uses WAL mode, `synchronous=FULL`, and `secure_delete=ON`. Purge performs checkpoint/truncation, `VACUUM`, and a second checkpoint. SQLite's documentation distinguishes active database content from WAL behavior and notes limits of secure deletion, including virtual-table shadow structures [11, 12]. We scan the actual files rather than treating a pragma as proof of erasure.

The filesystem adapter stores records in a JSON file using write-to-pending followed by atomic replacement. Both adapters persist a materialized index containing record IDs, content payloads, scope labels, and numerical vectors. The index is intentionally transparent so that a stale payload can be independently detected. It is not a Qdrant, Chroma, Mem0, or Letta implementation, and results must not be attributed to those products.

4.2 Retrieval and derivative construction

Retrieval uses a 128-dimensional hashed lexical vector. Tokens are lowercase ASCII alphanumeric/underscore matches; SHA-256 selects a dimension and sign, term counts accumulate, and vectors are normalized for cosine ranking. Tie-breaking uses the opaque record ID. This deterministic projection is not a learned embedding, does not model semantic paraphrase retrieval, and provides no embedding-inversion result.

In the storage study, each injected project source has three deterministic copies: a summary, cache entry, and checkpoint record. They preserve the target token and preference statement.

These fixtures deliberately guarantee a deletion dependency; they are not outputs of an LLM summarizer. The experiment therefore tests whether known copied content is removed and prevented from returning. It cannot estimate how often an actual summarizer creates such copies.

4.3 Validation and reproducibility

Eighteen implementation tests cover immediate purge, stale commit rejection, restore reconciliation, authority restart, repeated deletion, scope isolation, index consistency, and the two failing boundary controls. One test removes a derivative whose text does not contain the source token, verifying that scope selection does not rely solely on literal matching. Tests execute both adapters where appropriate.

The main run records protocol/code/data hashes, platform and SQLite versions, each trial's outcome, marker-containing active filenames, acquisition status, utility checks, and operation timings. Semantic agreement across adapters is audited separately from timing and physical-layout differences. Four processes execute disjoint episode shards in independent directories. Timing measurements therefore reflect a shared-host batch run, not isolated throughput or a production performance benchmark.

5 STORAGE-CONTRACT EXPERIMENTAL DESIGN

5.1 Data and sampling

LongMemEval [9] supplies 500 questions in the downloaded cleaned oracle release. We choose 20 questions from each of its six question-type fields using a fixed SHA-256 ordering over question identifiers, giving 120 episodes. All available oracle sessions for each chosen question become background raw records. Abstention items are not excluded from deletion tests; items without evidence do not contribute to gold-evidence recall.

LoCoMo [10] supplies ten conversation histories in the downloaded release. We choose two questions with nonempty answers and evidence per conversation using a fixed hash ordering, adding 20 episodes. All conversation sessions become background records for the selected question. These two questions per conversation are related, so their repetitions are not treated as 20 independent deployment populations.

Original session identifiers can carry answer text. We replace them with opaque record identifiers before indexing and keep their evidence mapping in evaluator-only metadata. The injected token is a deterministic 24-hex-character suffix of a hash over a study-specific prefix and episode ID. It is unique in this finite experiment but public and reproducible; no actual credential is inserted.

Each episode adds Project X with the target token and a route preference, plus an unrelated Project Y meeting-day control. Project X's target and all its prepared copies start at generation zero. The question's original background sessions provide corpus diversity and a retained-retrieval check; they do not turn the synthetic fault schedule into a representative real-world distribution.

5.2 Comparison arms

The source-only arm tests an intentionally incomplete operation and serves as a positive residue control. The more informative comparator is dependency purge, which removes all known active copies and shares the same storage cleanup. No comparison is made to the performance of a proprietary native deletion service.

Table 1. Six configurations. All use the same artifact state, compaction policy, and retrieval procedure. Source deletion is a reference adapter operation, not a claim about a vendor's scope-deletion API.

Configuration	Operation
Source deletion	Remove original record only; derivatives remain.
Dependency purge	Remove all active records with target scope once.
Full overlay	Purge, current authority, commit guard, restore reconciliation.
Without commit guard	Full overlay except stale writers are admitted.
Without restore guard	Full overlay except old snapshots are published unfiltered.
Without lineage purge	Revoke and guard future operations, but remove only the source immediately.

5.3 Lifecycle events and negative controls

Each configuration is tested under nine schedules: immediate observation; adapter restart; index rebuild; original-source reprocessing; pre-deletion snapshot restoration; delayed commit of a pre-deletion artifact; peer-copy synchronization with a newly minted ID; restore followed by stale commit; and stale commit followed by restart. Snapshot restoration replaces active state with the recorded old state unless the restore guard filters it. Reprocessing and sync submit an old-generation artifact through the managed commit path.

Two additional schedules apply only to the full overlay. Authority rollback replaces the deletion authority with its pre-deletion empty state and restores old memory. Adapter bypass writes a queued artifact while explicitly skipping the guard. These are expected failure controls: they challenge the assumptions required for revocation and must not be hidden in a favorable managed-event average.

The grid is $140 \times 2 \times 6 \times 9 = 15{,}120$ managed trials, plus $140 \times 2 \times 2 = 560$ boundary trials. The 15,680 trials are not 15,680 independent agents. Each trial starts by reinitializing the same episode state; read-only probes do not mutate it. Each processed episode verifies pre-deletion acquisition and retrieval so that an empty or never-learned target cannot pass deletion vacuously.

5.4 Measurements and hypotheses

For episode i, configuration m, backend b, and event e, define active record recovery

$$R_{imbe} = \mathbf{1}\{c_i \text{ occurs in an active artifact or index payload}\}. \tag{2}$$

The worst-event endpoint is $W_{imb} = \max_e R_{imbe}$ over the nine managed schedules. We also record target recovery in the top-three retrieval payloads, exact marker presence in persisted active files, literal project-preference residue, unrelated-control preservation, and unmodified background records.

Retained utility is measured as unchanged top-three ranking of raw records relative to the same background plus Project Y without Project X, and gold-session recall where an evidence mapping exists. This is a retrieval-preservation proxy, not generated-answer accuracy, personalization utility, or behavioral equivalence.

H1 predicts that a one-time purge can pass immediately and fail after an event. H2 predicts lower worst-event recovery with the full overlay. H3 predicts event-specific failure when its corresponding guard is removed. H4 predicts preservation of unrelated artifacts and matched retrieval. The protocol was saved locally before the main measurement; it was not registered with an external preregistration service. Runtime smoke tests preceded the full run and were used to address Windows file-sharing retries and caching, not to select favorable paper outcomes.

Table 2. Active record/index recovery by event, expressed as percentages over 140 episodes per backend. Both backends produce the same values. I: immediate; RS: restart; IX: rebuild; SP: source reprocessing; SR: snapshot restore; DC: delayed commit; PS: peer sync; SC: restore then commit; CR: commit then restart.

Configuration	I	RS	IX	SP	SR	DC	PS	SC	CR
Source-only deletion	100	100	100	100	100	100	100	100	100
Dependency purge	0	0	0	100	100	100	100	100	100
Full overlay	0	0	0	0	0	0	0	0	0
No commit guard	0	0	0	100	0	100	100	100	100
No restore guard	0	0	0	0	100	0	0	100	0
No lineage purge	100	100	100	100	0	100	100	0	100

5.5 Analysis scope

We report exact finite-grid counts and event-level patterns. A paired episode bootstrap with 10,000 resamples characterizes the observed worst-event contrast. Identical deterministic episode contrasts can produce a degenerate interval; that interval describes this test set and is not a bound on unseen deployment risk. Repeating the same target across events or backends does not create independent statistical evidence. A zero-event rate cannot establish universal forgetting.

6 STORAGE-CONTRACT RESULTS

All 15,680 trials acquired the target before deletion. Trial keys were unique, recorded code and protocol hashes matched, and semantic outcomes agreed across backends. The full overlay yielded 0/2,520 active-record recoveries, 0/2,520 persisted-byte recoveries, and 0/2,520 top-three retrieval recoveries. These counts concern the enumerated managed artifacts, not every storage surface of an AI service. Table 2 reports the deterministic event pattern.

6.1 Immediate success and later reappearance

Dependency purge passed immediate, restart, and index-rebuild checks but failed all 140 episodes under each of the other six event schedules, on each backend: 840/1,260 managed trials per backend (66.7%). All 140 episodes had at least one post-deletion recovery. Source-only deletion failed all nine schedules because surviving derivatives retained the target. A clean active-state observation is thus compatible with later failure. The source-reprocessing and peer-copy schedules create a new ID, showing why record-specific removal is insufficient when a writer can recreate old content under a different identity. The tested scope epoch supplies the relevant authorization boundary.

6.2 Guard ablations

Removing the commit guard caused recovery in five schedules (700/1,260 trials per backend); removing restore reconciliation caused recovery in two (280/1,260); removing lineage purge caused recovery in seven (980/1,260). The paired episode-level worst-event contrast between full overlay and dependency purge was -1.00 on each backend; its descriptive percentile bootstrap interval was $[-1.00, -1.00]$. This degenerate interval reflects identical controlled episode outcomes. These event-specific effects support the implementation mechanism under controlled interleavings. They do not imply that the same rates occur in ordinary deployments. The events deliberately exercise the behavior each guard is designed to prevent.

Table 3. Deletion operation timing from the shared-host batch run. Values include this prototype's persistence, cleanup, and index rebuild. They are informational, not isolated system-performance estimates.

Backend	Configuration	Median (ms)	95th percentile (ms)
SQLite WAL	Dependency purge	34.11	72.63
SQLite WAL	Full overlay	35.78	78.76
JSON files	Dependency purge	20.98	33.29
JSON files	Full overlay	22.59	37.99

6.3 Active files, external fixtures, and utility

In all 2,520 full-overlay managed trials, the unrelated project record and all background records were preserved, and background retrieval rankings equaled the no-target baseline. Mean gold-session recall at three was 0.8876 across 280 backend–episode observations. This is an absolute retrieval diagnostic, not an LLM answer-accuracy estimate or an independent sample of 280 conversations. Retained fault-replay fixtures contain the target throughout the experiment. The result is therefore a distinction between active-store absence and retained-copy recoverability, rather than a claim that all information is gone.

6.4 Operation cost

Median full-overlay deletion took 35.78 ms on SQLite WAL and 22.59 ms on JSON files in this shared-host run. The corresponding one-time-purge medians were 34.11 ms and 20.98 ms. Table 3 gives measured medians and 95th percentiles; these timings should not be generalized to remote stores or loaded production agents.

6.5 Boundary-control failures

Authority rollback and adapter bypass each restored the target in all 140 episodes on both backends: 560/560 boundary trials. All retained replay fixtures remained recoverable, including during successful active-store deletion. The authority-rollback result rules out a stronger interpretation in which the ordinary memory snapshot somehow proves its own currency. If the current authority is also reset, the prototype cannot distinguish an old authorization from a valid one. Adapter bypass likewise shows that a control plane must cover every publication path.

7 BEHAVIORAL AND REACHABLE-CACHE EXTENSION

7.1 Question, hypotheses, and fixed protocol

We ask a second question: *Does absence from managed persistent storage imply absence from reachable process caches or absence of influence when a caller retains a model-written summary?* The protocol records H5: derivatives and one-time purge followed by restore permit greater target-route selection than never-seen; H6: full restore with fresh caller context reproduces clean-world supplied evidence; H7: unmanaged live summaries can preserve influence despite clean storage; and H8: the original memoizer retains reachable plaintext keys until explicitly cleared.

This extension is a separate experiment, preserving all original code and trial results. Its 24 episodes comprise the first two selected LongMemEval examples per six question types and 12 fixed hash-ranked LoCoMo examples. A fixed 1,500-character excerpt of the first background record supplies distractor context. These are corpus-grounded synthetic tasks, not official benchmark QA scores. The corpus does not supply the artificial secret or preference.

Each episode has a unique reproducible 24-hex code, a balanced seed-shuffled ALPHA/BETA preference, and an independent Project Y meeting day. One actual model-generated summary receives X and Y facts; that summary inherits both scopes and is reused unchanged across paired arms. No episode is excluded for failed acquisition. The independent Y source is retained when the mixed-scope summary is removed.

7.2 Model, evidence conditions, and probes

Inference uses official Qwen2.5-1.5B-Instruct Q4_K_M [17], pinned at revision `91cad51170dc3469`, with llama.cpp release b11425, one local CPU serving slot, 4,096 context tokens, eight threads, temperature zero, and seed 20261006. Prompt caching and RAM prompt-cache retention are disabled. This configuration does not establish physical KV erasure. The 24 summary calls allow 160 output tokens; 720 probe calls allow 48. The manifest records full model/runtime hashes, decoding settings, source/input hashes, and all requests and responses.

The never-seen arm receives only background and Y. Sham deletion removes a neutral Z record while X remains; it is an operation control, not a length-matched neutral-content control. Source-only removes X but retains its model-written summary. Purge-plus-restore reintroduces a pre-deletion snapshot after one-time scope cleanup. Full-plus-restore filters that snapshot against current revocation and supplies fresh caller context. Full-plus-live-summary leaves the managed store clean but supplies the old summary as earlier conversation evidence. Managed operations use the JSON adapter and the cache-clearing subclass; inference receives all current records directly, so this extension is not a learned-embedding retrieval test.

Two direct probes request the code, one explicitly allowing notes. Two route probes reverse option order. They ask for the recorded preference, or the first option if none is recorded, and never require the model to state the code. A fifth probe checks the retained Y day. Route parsing accepts a valid JSON label or a unique unambiguous ALPHA/BETA label; malformed/ambiguous answers count as non-target choices and their rate is separately reported. Direct emission is a lower-bound exact-string measure. The primary endpoint is each episode's mean target-choice indicator across both route orders, paired with never-seen. Percentile intervals resample 24 episode clusters 10,000 times with a fixed seed; they are descriptive conditional intervals, not a population guarantee.

7.3 Generated-response results

All 744 primary planned inferences completed. The generated summary retained the exact code in 24/24 episodes; the preferred route label appeared in 24/24 summaries. Label presence alone does not establish correct preference semantics. Table 4 reports every episode, including acquisition failures. Full-plus-restore and never-seen each answered the retained day correctly in 19/24 episodes, versus 24/24 for sham deletion. Thus matching the clean world does not preserve the contaminated state's higher generated-answer utility: retaining the independent Y source is weaker than preserving every answer. There were 0 length-truncated probe responses, and the largest probe prompt had 603 tokens.

Full-plus-restore and never-seen supplied identical prompts in 120/120 comparisons and produced identical response text in 120/120. Prompt equality follows from the chosen evidence construction. It is a reproducibility check, not independent evidence of counterfactual trajectory reconstruction, distributional equivalence, or model-weight forgetting. Full-plus-live-summary contained no active target record in any episode, but caller-supplied target code remained in 24/24 episodes. That boundary condition measures whether old evidence supplied outside the managed adapter can influence responses.

Table 4. Actual model responses over 24 episodes per arm. Code is emission on either direct probe; target and valid route rates average both option orders; day is retained Y accuracy.

Evidence condition	Code	Target route	Valid route	Day
Never seen	0/24	0.500	1.000	19/24
Sham deletion	21/24	1.000	1.000	24/24
Source only	20/24	0.875	1.000	24/24
Purge + restore	21/24	1.000	1.000	24/24
Full + restore	0/24	0.500	1.000	19/24
Full + live summary	8/24	0.896	1.000	24/24

Table 5. Target-route differences relative to never-seen, with episode-cluster percentile intervals. Option-order trials are not independent samples.

Evidence condition	Paired difference	Descriptive 95% interval
Source only	+0.375	[+0.229, +0.521]
Purge + restore	+0.500	[+0.333, +0.667]
Full + restore	+0.000	[+0.000, +0.000]
Full + live summary	+0.396	[+0.229, +0.562]

7.4 Exploratory evidence-swap robustness check

After inspecting early primary-run outputs, we specified an additional check using all 24 episodes, without exclusions. In each of the two full-plus-live-summary route prompts, we swap whole-word ALPHA/BETA labels only in the evidence; the task's option order, private code, and unrelated facts stay unchanged. This is an explicitly post-pilot exploratory intervention, not an additional preregistered hypothesis test. It asks whether choices track evidence content rather than merely the presence of a longer note.

Across 48 additional inferences, the model selected the swapped preference in 42/48 cases, changed to the opposite route in 37/48, and produced a valid route in 47/48. Prompt token counts were identical in 0/48; the observed difference range was [-1, 1]. These responses test sensitivity to supplied evidence, not hidden-state erasure. The total model workload is 792 inferences, with primary and exploratory observations stored separately.

7.5 Reachable memoization-key remanence

The storage runner memoizes `vector(text)` with `functools.lru_cache`. Although cached values are numerical vectors, its keys retain the input strings. In a separate CPython-specific audit, we inspect directly reachable cache dictionaries with `gc.get_referents`; we do not scan arbitrary heap pages. Across 24 synthetic cases on each of two adapters, all 48 original full-overlay deletions leave a target-containing reachable key despite zero active-file matches. A subclass calling `vector.cache_clear()` after deletion removes such keys in all 48 paired cases. All 96 cases preserve the unrelated record and have clean active-file scans.

This result corrects an important coverage ambiguity: the original persistent-file result remains true, but cannot describe all process copies. Clearing references is not physical zeroization, and other holders may retain the same string. The cache fix clears unrelated memoized vectors too,

creating a potential recomputation cost; the persistent background records remain unchanged. We make no performance claim for this extension.

8 DISCUSSION

8.1 Revocation is a standing condition

A one-time purge changes the store at deletion time. An epoch guard changes whether subsequent state is admissible. These are complementary obligations: removing derivatives closes current copies, while commit and restore enforcement prevent some future copies. The no-lineage arm demonstrates why preventing later writes cannot repair an already retained derivative by itself.

This interpretation is consistent with the known building blocks in prior work and does not require calling an epoch novel. The practical value of the experiment lies in an explicit contract, cross-adapter tests, and visible failure assumptions. An overlay becomes useful only if actual integrations preserve those assumptions.

8.2 Deletion receipts and claim language

A receipt should identify scope, authority generation, managed adapters, operations, unresolved copies, and audit coverage. A successful active scan can justify a statement about the scanned files and methods. It cannot justify a statement about provider logs, model weights, unallocated disk sectors, or retained backups. A signature authenticates a receipt's author; it does not prove that every copy was erased.

Our measured fixture recoverability illustrates a necessary reporting distinction. A restore guard can achieve an active-use contract even when a retained old copy contains the information. A system promising complete recoverability elimination would need an additional policy for registered backups, key management, remote replicas, and underlying media. Those policies are not implemented here.

8.3 Scope lineage and semantic inference

Correct scope labels make a deterministic purge tractable. Missing or incorrect labels can cause both leakage and collateral removal. The present artifacts inherit known labels; there is no automatic semantic lineage detector. Real summaries mixing several projects require either whole-artifact removal or selective reconstruction. An adversarially incorrect label can evade this overlay.

Moreover, an unchanged retained record can imply a deleted fact. The exact-target copy test does not measure this inferential channel. The extension directly measures route selection and never-seen behavior, but uses only one greedy model, synthetic preferences, and an operation sham. Independently authored semantic preferences, length-matched shams, stochastic probes, and retained-correlate tests remain necessary for stronger behavioral generalization.

9 THREATS TO VALIDITY

Construct validity. The first study uses deterministic derivatives; its checkpoint is not a model-weight checkpoint or KV cache. The second uses generated summaries and actual responses, but does not reproduce a full autonomous agent trajectory. Retrieval uses a hashed lexical projection and payload inspection. Literal absence does not imply semantic non-influence. The term "agent memory" describes the external substrate under study, not a full autonomous agent execution.

Internal validity. The test coordinator serializes operations and injects specific interleavings. It does not validate a true asynchronous transaction protocol. Shared-host parallel execution and Windows file-sharing behavior affect timings. Both adapters use common overlay logic, so agreement supports adapter consistency rather than independent implementation replication. The

source-only positive control is weaker than a complete native scope-delete API; the dependency-purge arm is the meaningful one-time-cleanup baseline.

External validity. The corpus backgrounds are public conversational-memory examples, but the inserted projects, scope labels, and fault schedules are controlled. The experiment samples a small LoCoMo release and oracle rather than full LongMemEval-S histories. It does not represent all memory vendors, remote services, or user behavior. It estimates performance on a deterministic challenge grid, not natural fault incidence.

Forensic coverage. Active scans inspect reachable adapter files. They do not guarantee physical-media sanitization, page-cache removal, power-loss recovery, or remote erasure. The separate memoizer audit establishes plaintext-key reachability in the original implementation and reference removal after clearing, not heap erasure or neural inversion. The local inference server and operating system may retain physical buffers outside this contract. Replay fixtures and result provenance intentionally retain evaluative information.

Statistical validity. Multiple events reuse each episode, and two LoCoMo questions share a conversation. Finite-grid counts and descriptive bootstrap contrasts should not be promoted to population-level significance. The 24-episode extension has two route orders within episode, shared LoCoMo conversations, and one greedy quantized model. Its conditional bootstrap does not resolve between-conversation dependence. There is no powered behavioral equivalence or generated-answer noninferiority test; identical full/clean prompts cannot supply such evidence.

Novelty and publication status. Several closely related 2026 works are preprints or incompletely accessible. Broader independent searches and full-method review remain necessary before claiming priority. The manuscript is a research draft based on the reported experiment, not an accepted ACM publication or a certified deletion system.

10 DATA, ARTIFACTS, AND RESEARCH INTEGRITY

The source downloads retain revision-pinned URLs, SHA-256 values, sizes, and upstream license/card files. LongMemEval's downloaded release declares MIT; LoCoMo declares CC BY-NC 4.0. The experiment package preserves those distinctions. No real private credential is used and no attempt is made to identify individuals. The extension sends downloaded public background excerpts and synthetic targets only to a local inference server; no inference provider receives them. Its model card declares Apache 2.0.

The downloadable artifact contains the protocol, Python reference implementation, tests, raw trial JSONL, per-event and per-arm CSV summaries, analysis code, and validation reports. Corpus files remain in the companion data directory, and the small Overleaf package excludes their large payloads. Reproduction instructions fetch the recorded public revisions and verify hashes. A publication release should also provide a persistent repository or archival identifier; none is invented here.

AI assistance was used to develop code, analysis, and manuscript text. All numerical claims are generated from recorded executable outcomes and checked against the result files. Human authors must independently review attribution, experimental adequacy, and venue-specific requirements before submission. No author affiliation, funding award, ethics approval, DOI, acceptance, or review result is fabricated.

11 CONCLUSION

The controlled experiment distinguishes removing currently materialized copies from maintaining revocation during later operations. The full overlay prevented active recovery in 2,520 managed

trials across two backends, while one-time dependency purge failed under six of nine lifecycle schedules. The failures under authority rollback and adapter bypass expose the assumptions behind those results. Retained replay copies remain recoverable, so the appropriate claim is bounded active-memory revocation. The separate model-response extension adds behavioral measurements, while the cache audit demonstrates reachable process residue missed by active-file scans. Complete deletion requires an explicit policy for each copy and future admission path. Native-framework integration, multiple model families, stochastic behavioral audits, realistic concurrent transactions, and semantic inference remain needed before general deployment claims.

REFERENCES

[1] Lei (Rachel) Chen et al. 2026. Deployment-Time Memorization in Foundation-Model Agents. ICML MemFM workshop listing; arXiv:2606.10062v2. https://arxiv.org/abs/2606.10062.

[2] Dongxu Yang. 2026. Control-Plane Placement Shapes Forgetting: An Architectural Study of Agent Memory Across Thirteen System Configurations. arXiv:2606.15903v2. https://arxiv.org/abs/2606.15903.

[3] Jiabei Liu and Mingyang Li. 2026. Deletion Is Not Forgetting: Behavioral Residuals of Deleted Memories in Stateful LLM Agents. SSRN working paper, posted September 7. Abstract consulted. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=7428966.

[4] Kuan Wang and Chao Zhang. 2026. MemLeak: Diagnosing Information Leaks in Multimodal Agent Memory. arXiv:2606.29788v1. https://arxiv.org/abs/2606.29788.

[5] Department for Science, Innovation and Technology. 2026. Thematic Review and Gap Analysis on AI Security. Lancaster University commissioned review, July 10. https://www.gov.uk/government/publications/thematic-review-and-gap-analysis-on-ai-security.

[6] Rishav Chourasia and Neil Shah. 2023. Forget Unlearning: Towards True Data-Deletion in Machine Learning. In *Proceedings of ICML*, PMLR 202, 6028–6073. https://proceedings.mlr.press/v202/chourasia23a.html.

[7] Zihao Ding, Jun Huang, and Liang Dong. 2026. When Unlearning Fails: Reliable Data Deletion under Post-Training in Agent Networks. arXiv:2607.28829v1. https://arxiv.org/abs/2607.28829.

[8] Chen Shen. 2026. What Eviction Destroys: A Restore-Counterfactual Audit of Forgetting in Agent Memory. arXiv:2609.08279v1. https://arxiv.org/abs/2609.08279.

[9] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2024. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. arXiv:2410.10813; accepted at ICLR 2025 according to the official repository. https://github.com/xiaowu0162/LongMemEval.

[10] Snap Research. 2024. LoCoMo: Official Dataset and Code Repository. Dataset release used: `data/locomo10.json`; attribution retained in the companion artifact. https://github.com/snap-research/locomo.

[11] SQLite Project. n.d. PRAGMA Statements: secure_delete and wal_checkpoint. Official documentation, accessed October 4, 2026. https://www.sqlite.org/pragma.html.

[12] SQLite Project. n.d. Write-Ahead Logging. Official documentation, accessed October 4, 2026. https://www.sqlite.org/wal.html.

[13] Chao Yao, Yangbo Wei, Zhen Huang, Junhong Qian, Chenle Chen, Shaoqiang Lu, Chen Wu, and Lei He. 2026. Forgetting Without Restarting: Execution-State Unlearning for Stateful LLM Agents. arXiv:2609.04875v1. https://arxiv.org/abs/2609.04875.

[14] Wataru Washio. 2026. Structural Forgetting in AI Memory Systems: A Two-Phase Framework Unifying Index Blocking, Output Gating, Selective Power Gating, and Key Revocation. Preprint; indexed abstract/excerpt consulted. https://philpapers.org/rec/WASSFI.

[15] Harish Gaggar. 2026. MemGov: Policy-Governed Memory Lifecycle Management for Enterprise LLM Agents. *Journal of Intelligent Decision Making and Information Science* 3(11s), 2199–2214. https://jidmis.org/index.php/jidmis/article/view/3221.

[16] Suan Lee. 2026. Temporal Fields Are Not Temporal Correctness: Measuring Bitemporal and Deletion Semantics in Deployed Agent Memory. Research manuscript and Temvera artifact; repository revision `29a0fb66e4049698`. Publication acceptance not independently established. https://github.com/suanlab/temvera.

[17] Qwen Team. 2024. Qwen2.5-1.5B-Instruct-GGUF. Official model card and Apache-2.0 licensed model; pinned revision `91cad51170dc3469`. https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct-GGUF.

[18] Xuanze Chen, Xukang Xie, Wentao Fu, Jiajun Zhou, Shanqing Yu, and Qi Xuan. 2026. MemSecBench: Tracking Agent Memory Poisoning from Persistence to Consequence and Repair. arXiv:2607.27080v1. https://arxiv.org/abs/2607.27080.

A EXECUTABLE PROTOCOL AND REPRODUCTION

Use Python 3.12 with NumPy for analysis. From the companion outputs directory, run the unit tests, then the experiment, then analysis. The frozen reference results were produced in four shards; the provided runner supports a single-process reproduction and the artifact instructions describe merging its output. Exact non-timing outcomes should agree; timings are hardware and host-load dependent.

```
python -m unittest discover -s experiment -p test_overlay.py -v
python experiment/run_experiment.py --out experiment/reproduction
python experiment/analyze.py --results experiment/reproduction
```

The reference protocol SHA-256 begins `a978d6b90b5cf899`. Full SHA-256 values, source hashes, input hashes, and environment metadata are retained in the JSON manifests. Write fresh results to a separate directory rather than overwrite the reference observations.

B EXTENSION REPRODUCTION

Fetch the pinned model/runtime using the extension's acquisition script, start its localhost server with the recorded arguments, and run the cache audit, behavioral runner, and analyzer. The model weights are deliberately excluded from the ZIP because they are independently downloadable and hash-verified. Research prompts and responses intentionally retain synthetic targets outside the deletion contract. Original results are preserved in the storage-study artifact; do not overwrite reference observations when reproducing.

```
python behavioral-experiment/acquire_llm.py
python behavioral-experiment/cache_audit.py
python behavioral-experiment/run_behavior.py
python behavioral-experiment/analyze_behavior.py
python behavioral-experiment/run_robustness.py
```

The acquisition script expects this project layout, with the supplied corpora in the outputs data directory. Full server startup instructions and checksums are in the companion README and manifests.

C INTERPRETING THE RECORDED OUTCOMES

Table 6. Storage-study result fields and their boundaries. Behavioral and cache outcomes are reported separately in Section 7.

Recorded quantity	What it establishes
Record recovery	Exact target in active record/index payload.
Persisted-byte recovery	Exact target in currently reachable adapter files.
Retrieval at three	Exact target in the top-three hashed-lexical payloads.
Preference residue	Literal project-preference sentence remains; no LLM effect measured.
Unchanged background	Retained records are structurally equal to their initial content.
Gold-session recall	Fraction of evaluator-labeled evidence sessions in top-three raw results.
Fixture recovery	Pre-deletion replay copy still contains the target; erasure is incomplete outside active scope.